

Reviewer Pack — Minimal Index of Quasi-Polynomial Growth of Boolean Fourier ...

Entry c3177754b2f2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 21:28:21 UTC

Minimal Index of Quasi-Polynomial Growth of Boolean Fourier Coefficients over Tseitin Circuit Width

Entry ID: c3177754b2f2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 21:28:21 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics (Fourier Analysis) **Field B** (complexity object): Complexity Theory: Tseitin Circuit Width

Statement:

{‘one’: ‘For any Tseitin formula on an n -vertex graph, the minimal index of quasi-polynomial growth among the absolute values of its Boolean Fourier coefficients is at least $\theta(2^n / \log^2(n))$.’, ‘two’: ‘This index provides a lower bound on the Resolution proof length for that Tseitin formula.’, ‘three’: ‘For any n -vertex graph with vertex degree Δ , this index is $O(\log^{(\Delta+1)}(n))$.’}

Rationale (proposer’s reasoning):

{‘one’: ‘Boolean Fourier analysis has been used to study complexity classes and circuits, but its application to Tseitin formulas in the context of Resolution proof length is novel.’, ‘two’: ‘Quasi-polynomial growth of Fourier coefficients has previously been associated with certain complexity-theoretic properties, suggesting a potential link with Resolution lower bounds.’, ‘three’: ‘The constructive mapping from Tseitin circuit width to the index of Fourier coefficient growth provides a concrete method for computation.’}

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3d84c5c619a78e47

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given n -vertex graph, if the minimal index of quasi-polynomial growth among the absolute values of its Boolean Fourier coefficients is at least $\theta(2^n / \log^2(n))$ and within $O(\log^{(\Delta+1)}(n))$, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Fourier Analysis' AND 'Tseitin Circuit Width' AND Boolean Fourier Coefficients' - 'Algebraic Combinatorics' AND Tseitin Circuit Width AND quasi-polynomial growth - Boolean Fourier Coefficients AND lower bound Resolution proof length Tseitin Circuit Width

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def gaussian_elimination(A, b):
    n = len(b)
    for i in range(n):
        # Find pivot
        max_row = i
        for k in range(i+1, n):
            if abs(A[k][i]) > abs(A[max_row][i]):
                max_row = k
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]

        # Eliminate below pivot
        for k in range(i+1, n):
            factor = A[k][i] / A[i][i]
            for j in range(i, n):
                A[k][j] -= factor * A[i][j]
            b[k] -= factor * b[i]

    # Back substitution
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i+1, n))) / A[i][i]
    return x

def matrix_multiplication(A, B):
```

```

m, k, n = len(A), len(B), len(B[0])
C = [[0] * n for _ in range(m)]
for i in range(m):
    for j in range(n):
        for l in range(k):
            C[i][j] += A[i][l] * B[l][j]
return C

def boolean_fourier_coefficients(G, n):
    m = len(G)
    F = [0] * (1 << m)
    for i in range(1 << m):
        sign = 1
        for j in range(m):
            if i & (1 << j):
                sign *= (-1) ** G[j]
        F[i] = sign / math.sqrt(m)
    return F

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.randint(5, 30)
    degree = random.randint(1, 10)
    G = []
    for _ in range(n):
        neighbors = set()
        while len(neighbors) < degree:
            neighbor = random.randint(0, n-1)
            if neighbor != _ and neighbor not in neighbors:
                neighbors.add(neighbor)
        G.append(list(neighbors))

    Tseitin_formula = []
    for i in range(n):
        for j in range(i+1, n):
            Tseitin_formula.extend([i, j])

    F = boolean_fourier_coefficients(G, n)
    min_index = min(abs(f) for f in F if abs(f) > 0)

    conjecture_holds = min_index >= (2 ** n / math.log(n)) ** 0.5
    counterexample = "" if conjecture_holds else "min_index <  $\theta(2^n / \log^2(n))$ "

    return {
        "metric_name": "min_index",
        "metric_value": min_index,
        "instances_tested": n,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

```

```

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c10d3288.py", line 100, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c10d3288.py", line 80, in run_trial
    F = boolean_fourier_coefficients(G, n)
        ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c10d3288.py", line 57, in boolean_fourier_coefficients
    sign *= (-1) ** G[j]
              ~~~~~^~~~~~
TypeError: unsupported operand type(s) for ** or pow(): 'int' and 'list'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying whether the conjecture's conditions are met. | next: Investigate and fix the error in the test code to ensure it can run to completion and provide results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11765
2	preregistration	ollama_remote	glm4:latest	0	0	5694
3	novelty	ollama_remote	glm4:latest	0	0	4970
4	novelty	ollama_remote	glm4:latest	0	0	5747
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18243
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8737
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12097
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11655
9	judge	ollama_remote	glm4:latest	0	0	12318

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 91225 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/c3177754b2f2.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c3177754b2f2.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/c3177754b2f2.tar.gz` (if generated)